Computational Economics

Lab: Working with AI Agents

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lab Overview

- **Choose and direct an agent**
  - Model families, agent applications, and the harness
  - A task prompt and the tool execution loop
- **Manage the information it uses**
  - Context, tokens, compaction, and caching
  - Project instructions and reusable skills
- **Connect, check, and reuse its work**
  - External tools through MCP
  - Inspect the result and rerun the program

One task throughout: turn a CSV file into a temperature chart.

LLM Families and Developers

A **large language model (LLM)** processes context and generates text, code, or requests to perform actions.

- **GPT family** — developed by **OpenAI**.
  Examples: GPT-6 Astra, GPT-6.1 Sol, GPT-6 Luna.
- **Claude family** — developed by **Anthropic**.
  Examples: Claude Fable 5, Claude Opus 5.5, Claude Sonnet 5.5.
- **Gemini family** — developed by **Google DeepMind**.
  Examples: Gemini 4 Argon, Gemini 3.8 Flash.
- **Grok family** — developed by **xAI**.
  Examples: Grok 4.20, Grok 4.7.
- **DeepSeek family** — developed by **DeepSeek**.
  Examples: DeepSeek-V4-Pro, DeepSeek-V4.1-Flash.

Agent Applications: LLM + Harness

An **agent harness** supplies information to the model and executes its requested actions through **tools**, such as file readers and command runners.

$$\underbrace{\text{LLM}}_{\text{selects actions}} + \underbrace{\text{harness}}_{\text{runs tools and manages context}} = \text{AI agent}$$

- **Codex** (OpenAI): an agent for coding and project work.
- **Claude Code** (Anthropic): an agent that reads files, edits code, and runs commands.
- **Cursor Agent** (Cursor): an agent integrated into the code editor.
- **Gemini CLI** (Google): an agent used through the terminal.
- **GitHub Copilot** (GitHub): agent features for repository tasks and code review.

An application may support several models. Choose the application and the model separately where supported.

Specify the Task with a Prompt

A **task prompt** tells the agent what to do now. Give the input, desired output, constraints, and a way to check completion.

Our task: a temperature chart

The working folder contains `data.csv`, with columns `date` and `temperature_c`.

> *Create* `plot_temperature.py` *to plot daily temperature in date order. Label the axes Date and Temperature (Celsius), and save the chart to* `results/temperature.png`*.*
>
> *Keep the input data unchanged. Use the project Python environment. Run the script, inspect the chart, and check the dates, values, and units. Report the files changed and the command used.*

This gives the agent an observable target: a runnable script and a checked chart.

Reading: OpenAI, Prompt engineering.

From a Prompt to Tool Execution

The model requests an action; the harness runs the tool and returns its result for the next step.

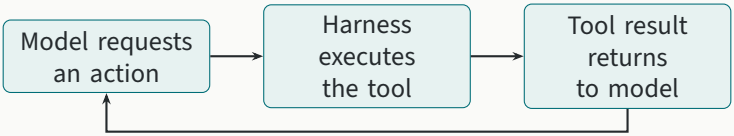


Figure 1: Request, execute, and use the result.

Chart task: read CSV → write script → run Python → inspect chart → revise if needed.

- A **CLI** accepts text commands, such as `python plot_temperature.py`. The **terminal** hosts the session; the **shell** interprets commands.

Use programs for repeatable rules; use the model for interpretation. Once checked, the plotting script can run again on new data.

Reading: CarpeDiem, Code is the most handy tool for AI.

The Prompt Is Part of the Context

Context is the input the harness supplies to the model for its next response.

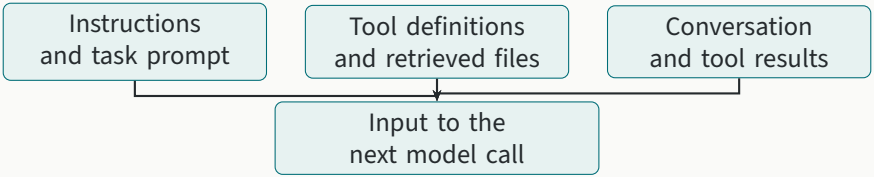


Figure 2: The harness assembles the model input.

- A **token** is a unit of input or output. The **context window** limits how much a call can accommodate.
- The harness loads relevant files and selects or summarizes earlier history.

"Fix the axis label" can accompany the CSV, script, and history: a short message can have a large input.

Keep the Working Context Relevant

Context engineering selects and maintains relevant information. Even a large context can contain details the model overlooks.

- **Load what helps.** For a plotting error, retrieve the relevant code and error message.
- **Keep related work together.** Revise the chart in one session; start a new one for unrelated work.
- **Preserve decisions.** **Compaction** replaces earlier detail with a shorter summary. Check that essential requirements survive.
- **Separate lengthy side tasks.** A **subagent** can investigate in a separate context and return a concise result.

A handoff to a fresh session

"The plotting script runs. Keep dates sorted and use Celsius. Next: compare the chart with `data.csv`."

Saved notes or **memory** help later sessions only when retrieved into their context.

Reading: CarpeDiem, Agent cost-saving tips (Part 1).

Context Caching Reuses Computation

Context caching reuses computation for an unchanged input prefix when a compatible cache is available.



Figure 3: Reuse A + B; process the new C.

- Eligible cached input can cost less to process. Retention and prices depend on the provider.
- Changing earlier instructions or history can invalidate reuse. Cached text still occupies context space.
- Cache expiry requires recomputation; compaction changes which information the model receives.

Judge the completed task: a high cache-hit rate alone does not imply low cost or good results.

Reading: CarpeDiem, Agent cost-saving tips (Part 2).

Reuse Instructions and Procedures

Keep recurring guidance in files the agent can load.

Project instructions
Rules for work in this project. Codex uses `AGENTS.md`; filenames and scope vary across applications.

Chart project rules

- Keep input data unchanged.
- Include units on axes.
- Save figures in `results/`.

A **skill**
A reusable procedure, with optional code and references. In the Agent Skills format:

```
plot-data/
  SKILL.md
  scripts/
  references/
```

Plotting procedure: read columns and units, create the figure, inspect it, and report checks.

The agent discovers a skill by its name and description, then loads its instructions when relevant. The skill guides tool use.

Reading: Agent Skills, Overview.

Connect External Tools with MCP

Suppose the CSV is stored in a shared cloud folder. The agent needs an authorized way to retrieve it.

Model Context Protocol (MCP) standardizes how an AI application communicates with servers exposing tools and context.

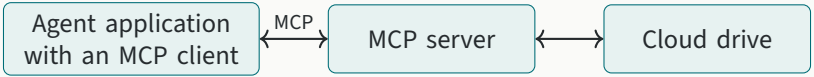


Figure 4: An external connection for the same chart task.

- A server can offer **tools** to act, **resources** to read, and **prompt templates** to reuse.
- A drive tool could retrieve the CSV; the agent then uses its existing plotting workflow.
- Local files can use the application's file tools directly. Add an MCP connection when a needed service provides one.

Reading: Model Context Protocol, Architecture overview.

Verify the Work Before Reusing It

A successful tool call or a confident completion message is evidence to inspect; compare the actual result with the task.

1. **Inspect the changes.** Review the script and confirm the input file was preserved.
2. **Reproduce the run.** Record the command and Python environment; rerun the script.
3. **Check the output.** Compare dates and values with the CSV; inspect axis labels and units.
4. **Keep the checked version.** Commit the script with Git so it can be reused or revised.

For temperatures of 20, 22, and 21 Celsius, the chart should preserve the date order and have its highest point at 22.

A **sandbox** restricts execution; **approvals** authorize actions. Retrieved files may contain **prompt injection**: instructions that try to redirect the agent. Treat external content as data to assess.

Practice: Run, Check, and Reuse

Create a working folder and save this practice file as `data.csv`:

```
date,temperature_c
2026-10-01,20
2026-10-02,22
2026-10-03,21
```

1. Choose an agent application and a supported model. Give it the chart prompt and access to this folder.
2. Follow one tool call: inspect what it reads or runs and the result returned to the model.
3. Review the script, rerun it, and check the chart against the CSV.
4. Add one observation to the practice CSV yourself; rerun the same script.

Extend the workflow: save a project rule for axis units, or package the plotting procedure as a skill when supported.

References and Further Reading

1. **OpenAI, Agent and Codex documentation. [Documentation]**
   - Codex as a platform: build on the open agent harness
   - Prompt engineering
   - Prompt caching
2. **Anthropic, Claude model and Claude Code documentation. [Documentation]**
   - Claude Code: Overview
3. **Microsoft, Visual Studio Code: agent and terminal documentation. [Documentation]**
   - Understand agent harnesses
4. **Model Context Protocol contributors, Model Context Protocol documentation. [Documentation]**
   - Architecture overview

References and Further Reading

5. **Anthropic (2025), Effective context engineering for AI agents. [Technical article]**
   - Context engineering vs. prompt engineering
6. **Agent Skills contributors, Agent Skills documentation. [Documentation]**
   - Agent Skills Overview
7. **CarpeDiem (2026), AI Learning from Zero: practical agent tutorials. [Video series]**
   - Code is the most handy tool for AI
   - Agent cost-saving tips (Part 1): tokens and context
   - Agent cost-saving tips (Part 2): context caching